

Compiler

MID CSE-3527

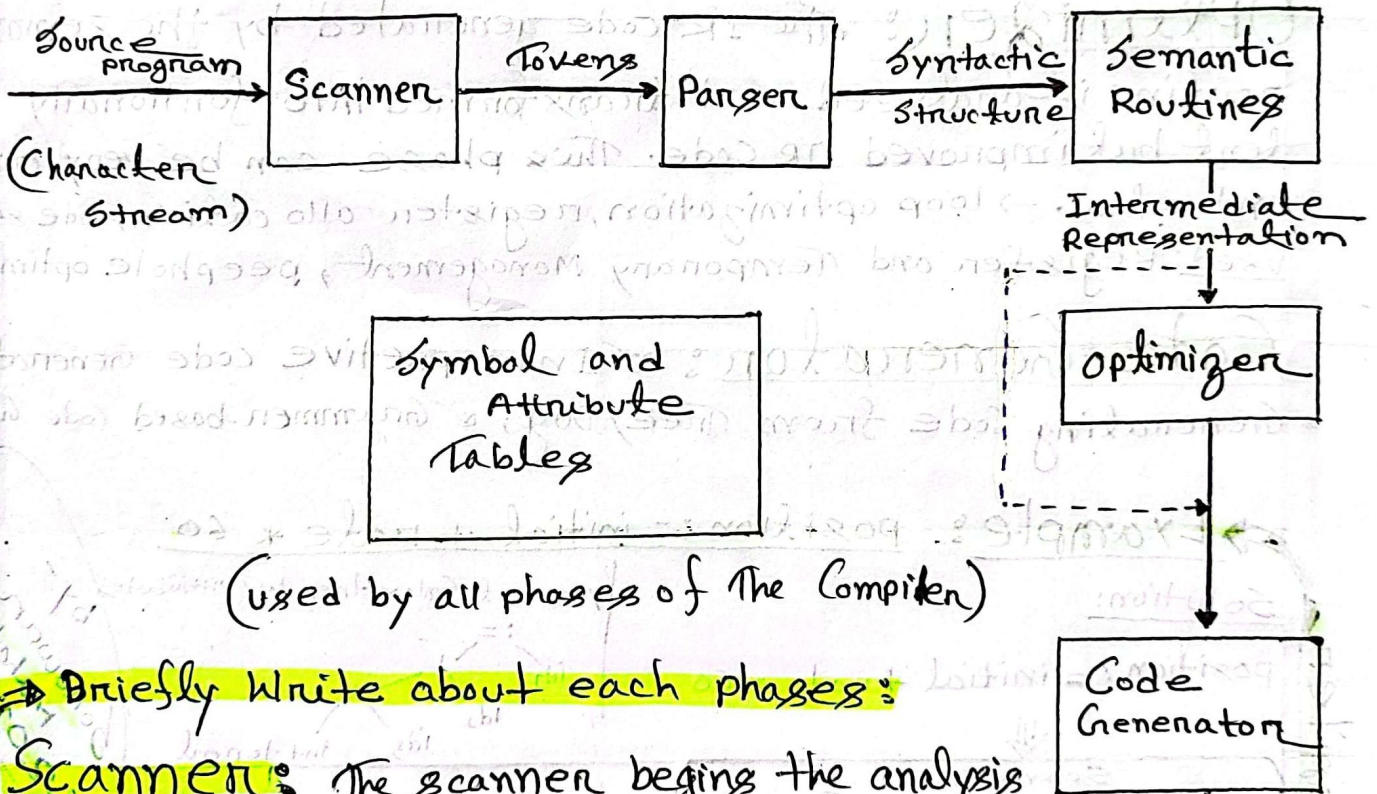
Note [without examples]

Prepared by
Sonowar Mahabub
Cell: 01521564157
► github.com/sonowar-cse

⇒ What is Compiler?

Ans: A compiler is a computer program that changes source code written in a high-level language into low-level machine language. It translates the code written in one programming language to some other language without modifying the definition of the code.

⇒ Structure of Compiler.



⇒ Briefly write about each phases:

Scanner: The scanner begins the analysis of the source program by reading the input character by character, and grouping characters into individual words and symbols, called Tokens.

Used → RE (Regular Expression), NFA, DFA (Deterministic Finite Automata), LEX.

Parser: Give a formal syntax specification (typically as a context free grammar [CFG]), the parser reads tokens and groups them into units as specified by the

productions of the CFG being used.

As syntactic structure is recognized, the parser either calls corresponding semantic routines directly or build a syntax tree.

Used: CFG, BNF (Backus-Naur Form), GAA (Grammar Analysis Algorithms), LL, LR, SLR, LALR parsers, YACC.

Semantic Routines: It perform two functions.

- check the static semantics of each construct
- do the actual translation.

It is the heart of a compiler.

Used: Syntax Directed Translation, Semantic Processing Techniques, IR (Intermediate Representation).

Optimizer: The IR code generated by the semantic routine is analyzed and transformed into functionally equivalent but improved IR code. This phase can be very complex and slow. $\rightarrow$ loop optimization, register allocation, code scheduling
used: Register and Temporary Management, peephole optimization.

Code Generator: $\bullet$ Interpretive code Generation, $\bullet$ Generating Code from Tree/Dag, $\bullet$ Grammar-based Code Generation

Example: $position := initial + rate * 60$

Solution:

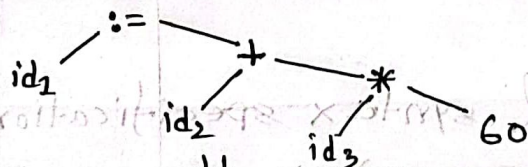
$position := initial + rate * 60$

Scanner
[Lexical Analyzer]

Tokens:
 $id_1 := id_2 + id_3 * 60$

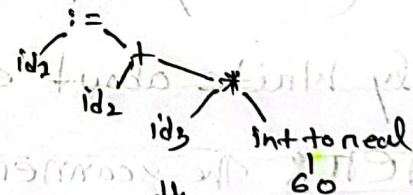
Parser
[Syntax Analyzer]

parse tree:



Semantic Process
[Semantic Analyzer]

Abstract Syntax tree w/ Attributes



Code Generator
(Intermediate Code Generator)

temp1 := int to real (60)
temp2 := id3 * temp1
temp3 := id2 + temp2
id1 := temp3

Non-optimized
Intermediate
Code

Code Optimizer

temp1 := id3 * 60.0
id1 := id2 + temp1

Optimized
Intermediate
Code

Structure of Compiler

Sonwan
Mahabub
C202032

Code optimizer

MOVF id₃, R₂ Target Machine Code
 MULF #60, 0, R₂
 MOVF id₂, R₁
 ADDF R₂, R₁
 MOVF R₁, id₁

Segment-02

by Sorowan Mahabub
 C201032

→ A compiler translates the entire source code in a single run.

→ An interpreter... line by line.

→ Interpreter is slower than Compiler.

Interpreter: -- is a program that executes the programming code directly instead of just translating it into another format.

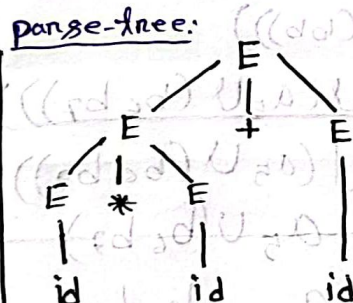
3 A derivation is a sequence of productions. $S \rightarrow \dots \rightarrow \dots$

3 Left-most Derivation: In this derivation, the I/P is scanned and replaced with the production rule from left to right. So, we have to read I/P string from left to right.

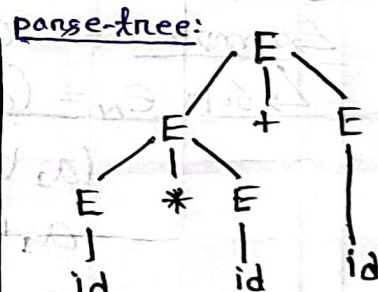
3 Right-most Derivation: In this derivation, the I/P is scanned and replaced with the production rule from right to left. So, we have to read the I/P string from right to left.

Q1 Grammar: $E \rightarrow E + E \mid E * E \mid (E) \mid id$ String: $id * id + id$

E
 $\rightarrow E + E$
 $\rightarrow E * E + E$
 $\rightarrow id * E + E$
 $\rightarrow id * id + E$
 $\rightarrow id * id + id$



E
 $\rightarrow E + E$
 $\rightarrow E + id$
 $\rightarrow E * E + id$
 $\rightarrow E * id + id$
 $\rightarrow id * id + id$



3 Parse Tree: A parse tree contains the following properties:

- ① The root node is always indicating start symbols.
- ② The derivation is read from left to right.
- ③ The leaf node is always terminal nodes.
- ④ The internal nodes are always non-terminal node.

3 Context-Free Grammar (CFG): It is a formal grammar which is used to generate all possible patterns of string in a given FL. It can be defined by 4 tuple as-
 $G(V, T, P, S) \rightarrow G$ is grammar, consists of a set of production rule
 $T \rightarrow$ is the final set of terminal symbol (denoted by lower case)
 $V \rightarrow$ is the final set of non-terminal symbol (denoted by capital letters)

$P \rightarrow$ is the set of production rules. $S \rightarrow$ is the start symbol used to derive the string.

Constructing CFG:

$\rightarrow L = \{w \in \{a,b\}^* \mid w \in L\}$
 $\text{Sol: } \{a^n b^n \mid n \geq 1\}$

The grammar could be -

$S \rightarrow a S b$	$L = \{a^n b^n \mid n \geq 1\}$
$S \rightarrow b S b$	$= \{a b b b, a a b b b \dots\}$
$S \rightarrow \epsilon$	Ans: $S \rightarrow a S b b b$
	$S \rightarrow a b b$

Regular Language

and expression

R.L are the simplest family of formal language. It can be defined in different ways -

- algebraically.
- by means of generating grammars.
- by means of recognizer machines.

Reg. expression is a string r over the letters of the terminal alphabet Σ , and containing a few meta symbols ($\epsilon, \cup, *, ()$) according to the following rules (s and t represents reg. expression).

- ① $r = \emptyset$ ② $r = a, a \in \Sigma$ ③ $r = (s \cup t)$ ④ $r = (s \cdot t)$ or st ⑤ $r = (s)^*$
- Precedence of operators: $\text{Star} (*)$, $\text{Concatenation}(\cdot)$, $\text{Union}(\cup)$ $\boxed{E = \emptyset^*}$

Sub-expression of Reg. - Expression:

- ① Consider a regexp. that is fully parenthesized.
- ② Number every alphabetical symbol occurring in the regexp.
- ③ Isolate the subexpression and put them into evidence.

Example: $e = (a \cup (b b))^* (c^+ \cup (a \cup (b b)))$

$\text{Sol: } e_n = (a_1 \cup (b_2 b_3))^* (c_4^+ \cup (a_5 \cup (b_6 b_7)))$

$(a_1 \cup (b_2 b_3))^* \quad c_4^+ \cup (a_5 \cup (b_6 b_7))$

$a_1 \cup (b_2 b_3) \quad c_4^+ \quad a_5 \cup (b_6 b_7)$

$a_1 \quad b_2 b_3 \quad c_4 \quad a_5 \quad b_6 b_7$

$b_2 b_3$

$b_6 b_7$

3 Closure of Reg. Family: $\rightarrow$ Reg contains all finite language. $\rightarrow$ REG is closed with respect to Concatenation($\cdot$), Union($\cup$), Star($*$)

REG $\subset$ LIB

3 A regexp f is Ambiguous if and only if, the corresponding marked regexp f generates two marked strings x and y such that if the indices are removed, the underlying ordinary strings are identical.

Ex: $(a \cup b)^ a (a \cup b)^* \Rightarrow a a (a \cup b)^* \Rightarrow a a \epsilon \Rightarrow a a$
 $\Rightarrow \epsilon a (a \cup b)^* \Rightarrow \epsilon a a \Rightarrow a a$

by -
 Sonowar Mahubub
 C202032
 Cell: 02523564257
 github.com/sonowar

3 Firstly read the pdf named 3.2

Link or Read in Brief Here

Segment-03
by Jorowar Mahabub
201032

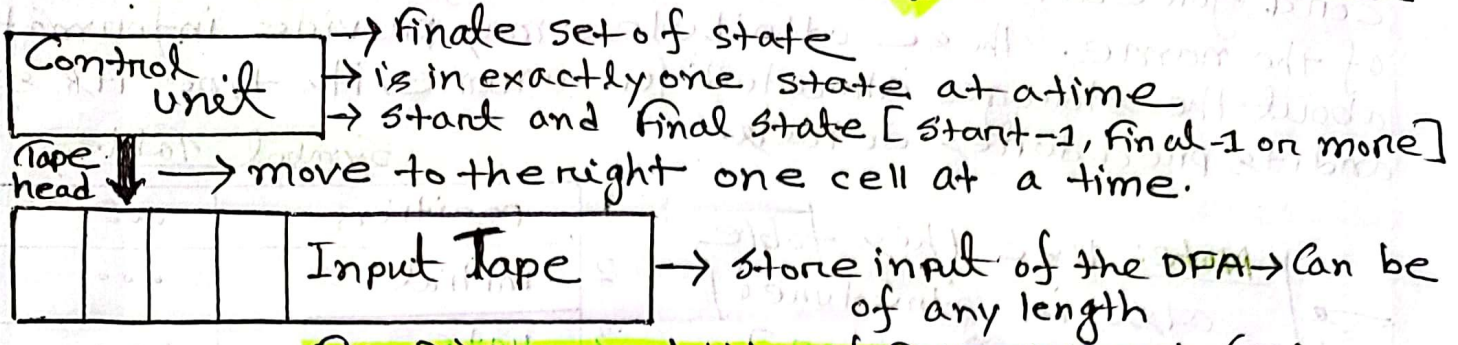


Fig: Diagram and Notes of finite Automata (FA)

FA → Read an input string from Tape. → Determine if the input string is in a language. → Determine if the answer for the problem is 'Yes' or 'No' for the given input on the tape.

How does FA work? → At the beginning,

- an FA is in the start state (initial state)
- its tape head points at the first cell

For each move, FA

- reads the symbol under its tape head
- changes its state according to the transition function to the next state determined by the symbol read from the tape and its current state.
- move its tape head to the right one cell.

FA Stop working -

- when it reads all the symbols on the tape.
- then it gives an answer, if the input is in the specific language:

Answer: 'YES' if its last state is a final state, otherwise 'NO'.

DFA, NFA's Definition:

NFA, DFA can be defined using a 5-tuple are $(Q, \Sigma, \delta, s, F)$, where -

- a set of state Q is a finite set.
- an alphabet Σ is a finite, non-empty set.
- a start state s in Q
- a set of final state F contained in Q

HEY,
NFA, DFA
is formal def.
Same, only Transition
function (δ) is different!

That's why it is
called finite
Automation (FA)

DFA - a transition function δ is a function: $Q \times \Sigma \rightarrow Q$

NFA - a transition function δ is a function: $Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$

3 Symbol Table Management [STM]:

The symbol table is a data structure containing a record for each variable name, with fields for the attributes of the name. These attributes may provide information about the storage allocated for a name, its type, its scope and its procedures details.

Symbol Table

1	position	...
2	initial	...
3	rate	...
4		

3 | Note: Use this table also in structure of Compiler [page-02]

Rules for Regular Expressions:

Let π, s, t be reg. expressions over $\{0, 1\}$.

$$\emptyset^* = \epsilon, \epsilon^* = \epsilon, (\pi + \epsilon)^+ = \pi^*$$

$$\pi^* = \pi^* (\pi + \epsilon) = \pi^* \pi^* = (\pi^*)^*, (\pi^* s^*)^* = (\pi + s)^*$$

Pumping Lemma: Let L be a regular language. Then there exists an integer $n \geq 0$ such that for every string x in L that $|x| \geq n$, there are strings u, v, w such that

- $x = uvw$

- $v \neq \epsilon$

- $|uv| \leq n$ and

- for all $k \geq 0$, $uv^k w$ is also in L .

by -

Sonwan, Mahabub
C201032

Cell: 015221564357

that's all for theory. Just warm-up with examples from others source or 3.1 exercise.

$$\emptyset \leftarrow S \times \emptyset : \text{no transition}$$

$$S \leftarrow (\{ \emptyset \} \cup S) \times \emptyset : \text{no transition}$$